

UMBRELLA OS

DATA SOURCING + BENCHMARK GUIDE

Computational biology data acquisition, provenance, benchmark construction, AMR annotation, scalable processing, and optional foundation-model layers.

Purpose: This document is the companion data guide for the Umbrella OS build. It tells the team where the scientific data comes from, which sources are authoritative, how to obtain and normalize the data, how to structure the benchmark, and where advanced models fit.

Important: The challenge brief says organizers will provide a fixed benchmark dataset. Treat that pinned dataset as the event benchmark of record. External datasets are for development, augmentation, stress-testing, and research exploration unless the organizers explicitly allow otherwise.

Verified sources reviewed for this guide: BV-BRC, NCBI AMRFinderPlus, ResFinder, cAMRah, XTree, HyenaDNA and DNABERT-2.

1. Data Strategy in One Page

What data Umbrella OS actually needs

Umbrella OS should not begin by downloading hundreds of gigabytes. Begin with the benchmark schema, then expand the data footprint only when a feature needs it. The central machine-learning task is supervised prediction from bacterial genome-derived features to laboratory-measured antibiotic outcomes.

The preferred training row is a **genome-antibiotic observation**. It should link a stable genome identifier, the biological sequence, the antibiotic being tested, the laboratory method, the raw measurement where available, the testing standard, and the final phenotype label. BV-BRC exposes these fields through its genome and genome_amr data APIs.

Core rule: Separate measured phenotype from computed genotype annotations. Laboratory results are labels. AMRFinderPlus/ResFinder/cAMRah outputs are features or independent annotations. Do not train on a prediction field and then call it ground truth.

Layer	Primary source	Use in Umbrella OS
Genome sequence	BV-BRC / organizer dataset	FASTA sequence and genome identity
Laboratory phenotype	BV-BRC genome_amr / challenge dataset	MIC / phenotype ground truth
AMR annotation	NCBI AMRFinderPlus	Gene + known point mutation features
Cross-check annotation	ResFinder	Independent AMR gene / mutation evidence
Ensemble workflow	cAMRah	Harmonized multi-tool comparison
Large-scale k-mer processing	XTree	Optional fast reference / coverage layer
Advanced representation	HyenaDNA / DNABERT-2	Optional stretch features, not baseline
Tutorial mirrors	Kaggle	Only when original source and license are traceable

2. Organizer Dataset: Use This First

Event benchmark and evaluation discipline

The supplied challenge brief states that the organizers will provide a fixed challenge dataset built around real bacterial genome data with laboratory antimicrobial susceptibility results. It describes 15,000+ bacterial genomes linked to laboratory outcomes, including quantitative information such as minimum inhibitory concentration (MIC), and instructs teams to use laboratory-measured results rather than general phenotype fields that may contain model-generated predictions.

For the hackathon: Treat the organizer-pinned split as immutable. Do not leak labels between train and test. Do not use external predictions to reconstruct the official test labels. Record the exact file names, release date, hash and schema received from the organizers.

Recommended benchmark manifest:

```
dataset_manifest.json
{
  "source": "Organizer challenge dataset",
  "received_at": "YYYY-MM-DDTHH:MM:SSZ",
  "genome_count": 15000,
  "phenotype_source": "laboratory_measured",
  "train_split": "organizer_defined",
  "test_split": "organizer_defined",
  "sha256": "<file hash>",
  "schema_version": "<provided version>"
}
```

Store the manifest inside Umbrella Core. Every model run should reference the manifest ID so that a prediction can be traced back to the exact benchmark snapshot.

3. BV-BRC: Primary Genome + Phenotype Source

Bacterial and Viral Bioinformatics Resource Center

BV-BRC is the strongest primary source for the project because it exposes public bacterial genomes alongside structured metadata and AMR phenotype records. Its API documentation shows that genome records carry stable genome IDs, assembly accessions and evidence fields, while genome_amr records include antibiotic, laboratory testing method, raw measurement text, parsed measurement values, measurement units, phenotype, testing standard and provenance.

The BV-BRC FTP documentation also provides public genome files by genome ID, including FASTA contigs (.fna), protein sequences (.faa), feature tables, GFF, and specialty-gene assignments. Large-scale downloads are intended to use encrypted FTPS.

Best use: Use BV-BRC for the actual sequence + phenotype join. Keep the phenotype evidence field explicit so that laboratory measurements are never confused with computational predictions.

Useful endpoints and files:

Resource	Purpose
https://www.bv-brc.org/api/doc/genome	Genome metadata schema and API examples
https://www.bv-brc.org/api/doc/genome_amr	AMR phenotype schema and API examples
https://www.bv-brc.org/api/doc/genome_sequence	FASTA / sequence API formats
https://www.bv-brc.org/docs/quick_references/ftp.html	FTPS structure and bulk-download guidance
https://www.bv-brc.org/	Interactive genome/phenotype browsing

4. BV-BRC Acquisition Workflow

Start small, then scale

For a solo 18.5-hour build, begin with the organizer files or a small BV-BRC sample. Build the pipeline so that the same code can later consume the larger corpus.

Bulk genome download can be driven from a `genome_list` file. BV-BRC currently documents FTPS access and an example `wget` pattern for downloading `.fna` files by genome ID.

Example shell workflow:

```
# 1) Create a list of genome IDs from the benchmark or filtered BV-BRC export
cat > genome_list <<'EOF'
511145.12
<genome_id_2>
<genome_id_3>
EOF

# 2) Download assemblies over FTPS
while read -r id; do
  wget -qN "https://ftp.bv-brc.org/genomes/$id/$id.fna"
done < genome_list

# 3) Keep raw data separate from derived data
mkdir -p data/raw/bvbrc data/derived/amr data/benchmark
```

For a large corpus, do not download blindly. Build a manifest first, download only the selected genome IDs, verify file sizes/checksums when available, and log every transfer. The goal is deterministic re-runs, not a one-off pile of FASTA files.

Storage rule: Raw data is read-only. Derived data goes elsewhere. Never edit a downloaded FASTA in place.

5. Benchmark Schema and Label Construction

The core table Umbrella ML will train on

Build one normalized observation table with one row per genome-antibiotic test. This lets the same table drive the FP-style dashboard, model training, model evaluation, and scientific audit trail.

Field	Example	Why it matters
genome_id	511145.12	Stable join key
assembly_accession	GCF_000005845.2	Sequence identity/version
taxon_id	562	Taxonomy grouping
organism	Escherichia coli	Human-readable organism
antibiotic	Ciprofloxacin	Prediction target context
measurement_value	0.25	Numeric MIC or parsed value
measurement_sign	< / > / =	Censoring semantics
measurement_unit	ug/ml	Unit normalization
resistant_phenotype	Resistant	Categorical label when provided
testing_standard	CLSI/EUCAST	Breakpoint interpretation
testing_standard_year	2024	Reproducibility
laboratory_typing_method	Broth microdilution	Method context
phenotype_evidence	Phenotype	Prevents label leakage

Keep the raw measurement and the derived label. If the challenge provides a categorical phenotype directly from the laboratory record, use it as the classification label. If the task asks for MIC regression, retain the raw numeric field and the sign. Do not silently convert censored MICs to exact values.

6. AMRFinderPlus: Default Resistance Annotation

NCBI reference genes + HMMs + curated cutoffs

NCBI describes AMRFinderPlus as a tool that identifies acquired AMR genes and known resistance-associated point mutations from protein annotations and/or assembled nucleotide sequences. NCBI makes the software and databases freely available and uses the tool in the Pathogen Detection pipeline.

Its detection stack combines sequence alignment and profile-based evidence with the NCBI Reference Gene Catalog, Reference Gene Hierarchy, HMM catalog, curated cutoffs and a tuned ruleset. This makes it the preferred default annotation source for Umbrella OS.

Important provenance: Store the AMRFinderPlus software version and database version with every annotation batch. A gene call without the database snapshot is not fully reproducible.

Official resources:

Resource	Link / purpose
NCBI AMRFinderPlus	https://www.ncbi.nlm.nih.gov/pathogens/antimicrobial-resistance/AMRFinder/
NCBI AMR Resources	https://www.ncbi.nlm.nih.gov/pathogens/antimicrobial-resistance/resources/
Source repository	https://github.com/ncbi/amr

Feature design: create binary, count, or sparse features from AMRFinderPlus outputs. Examples include presence of a named AMR gene, presence of a known resistance-associated mutation, and count of resistance markers by antibiotic class. Keep raw annotations in a provenance table instead of flattening everything into an opaque JSON blob.

7. ResFinder and cAMRah: Independent Evidence

Cross-validation instead of blind trust

ResFinder is a complementary AMR detection resource for acquired resistance genes and chromosomal mutations. Use it as an independent annotation stream rather than automatically merging it into AMRFinderPlus.

cAMRah provides a reproducible workflow that combines six AMR-finding tool/database combinations and harmonizes their outputs. The published workflow is Docker- and WDL-oriented and was designed to improve reproducibility and cross-tool comparability.

Umbrella approach: AMRFinderPlus is the default. ResFinder is the cross-check. cAMRah is the optional ensemble/harmonization layer when you need to compare tool disagreement.

Tool	Role	Output Umbrella should keep
AMRFinderPlus	Primary AMR annotation	gene, allele/mutation, method, version, evidence
ResFinder	Independent cross-check	gene/mutation hit, identity/coverage fields, database version
cAMRah	Harmonized ensemble	per-tool calls, consensus/harmonized gene label, locus mapping

Source links: ResFinder - <https://genepi.food.dtu.dk/resfinder> ; cAMRah paper - <https://pmc.ncbi.nlm.nih.gov/articles/PMC12910510/> ; cAMRah repository - <https://github.com/JCVenterInstitute/CAMRA>

8. XTree: Scale Layer for K-mer / Genome Search

When the dataset becomes too large for naive alignment

XTree is a k-mer-based sequence lookup and tallying tool intended for efficient metagenomic and large-reference analysis. Its documentation describes genome coverage analysis, annotation cross-tabulation, per-query statistics, compressed databases, and multithreading. It can operate over large reference sets and supports FASTA/FASTQ inputs.

For Umbrella OS, XTree should not replace the baseline ML pipeline. It is an optional scale layer for sequence search, coverage, reference matching, and fast pre-screening when you want to demonstrate that the platform can move from a 15k-genome benchmark toward a much larger corpus.

Official repository: <https://github.com/two-frontiers-project/2FP-XTree>

```
# Conceptual pipeline (not a benchmark command)
Reference FASTA
  |
  v
XTree BUILD --k 29 --comp 2
  |
  v
XTree database (.xtr)
  |
  v
Query FASTA / FASTQ
  |
  v
ALIGN -> coverage / per-query / taxonomy outputs
```

Engineering note: Keep the XTree database external to your transactional database. Treat it as a versioned scientific index with its own build manifest.

9. Model Stack: Baseline First, Foundation Models Second

The model that must ship

The recommended baseline is one regularized logistic-regression model per antibiotic, trained on interpretable genome-derived features. This is fast on CPU, easy to calibrate, and gives a defensible reference against which advanced models can be measured.

Model layer	Input	Compute	Use
Baseline LR	AMRFinderPlus features + selected genomic metadata	CPU	Official benchmark baseline
Tree / boosting stretch	Same engineered features	CPU	Non-linear benchmark comparison
DNA embeddings	Selected sequence regions/chunks	GPU preferred	Advanced representation
Hybrid model	AMR annotations + embeddings	GPU preferred	Compare against baseline

Avoid feeding an entire bacterial genome into a Transformer by default. Long bacterial genomes are too large for many standard attention pipelines. HyenaDNA is specifically designed for long-range genomic modeling and supports contexts up to 1 million tokens at single-nucleotide resolution; its official project notes different checkpoint lengths and GPU requirements. DNABERT-2 is a 117M-parameter foundation model using BPE tokenization and ALiBi, with benchmark coverage across multiple species/tasks.

10. Optional Deep-Learning Stretch

How to use genomic foundation models without blowing the sprint

Use the foundation models as an experimental branch, not the project backbone. The safest hackathon design is to extract embeddings from selected, annotated DNA regions or fixed-size genome chunks, aggregate those vectors into one sample representation, and compare performance against the logistic baseline.

HyenaDNA: official repository <https://github.com/HazyResearch/hyena-dna> . The project states that its pretrained models cover contexts up to 1 million tokens at single-nucleotide resolution and provides Colab / Hugging Face paths for inference. DNABERT-2: https://github.com/MAGICS-LAB/DNABERT_2 and the 117M checkpoint referenced in that repository.

```
# Recommended stretch architecture
Genome FASTA
|
+--> QC / chunking
|
+--> select annotated regions
|
+--> embedding model
|
+--> mean/max/attention pooling
|
+--> antibiotic-specific classifier
|
+--> compare with LR baseline
```

Do not train a new foundation model during the challenge. Use pretrained checkpoints only if GPU access is available. Free Colab GPU availability is not guaranteed, so keep this branch optional and time-boxed.

11. Recommended Umbrella Data Lake Layout

Make provenance a first-class feature

```
data/
├── raw/
│   ├── organizer/
│   ├── bvbrc/
│   ├── amrfinder/
│   ├── resfinder/
│   └── external/
├── manifests/
│   ├── organizer_v1.json
│   ├── bvbrc_snapshot.json
│   └── annotation_runs/
├── derived/
│   ├── fasta_normalized/
│   ├── qc/
│   ├── amr/
│   ├── variants/
│   ├── features/
│   └── labels/
├── indexes/
│   ├── trie/
│   ├── bloom/
│   └── xtree/
└── benchmark/
    ├── train.parquet
    ├── validation.parquet
    ├── test.parquet
    └── metrics/
```

Every derived artifact should carry a `source_manifest_id`, `tool_version`, `database_version`, `created_at` timestamp, `command/config` hash, and `upstream input` hash. This is how Umbrella OS becomes a scientific operating system rather than a folder full of scripts.

12. End-to-End Data Pipeline

From public genome to Umbrella prediction

```
1. Acquire
   organizer dataset / BV-BRC
   |
2. Validate
   schema + file integrity + IDs
   |
3. Normalize
   FASTA headers + metadata + units
   |
4. QC
   length / GC / ambiguous bases / duplicates
   |
5. Annotate
   AMRFinderPlus (default)
   |
6. Cross-check (optional)
   ResFinder / CAMRaH
   |
7. Feature build
   gene presence + point mutations + metadata
   |
8. Split
   organizer-defined or leakage-safe train/val/test
   |
9. Train
   one regularized LR per antibiotic
   |
10. Calibrate + evaluate
    AUROC / AUPRC / Brier / calibration / confusion matrix
    |
11. Serve
    FastAPI inference service
    |
12. Visualize
    Umbrella OS apps + scientific report
```

The same pipeline should operate in batch mode and interactive mode. A researcher can select a sample inside Sample Vault, while the benchmark runner can process thousands of genomes without opening the desktop UI.

13. Benchmark Quality, Leakage and Metrics

How to avoid a beautiful but invalid result

The biggest scientific risk is not an incorrect UI; it is leakage or mislabeled evidence. Before adding deep learning, establish a clean benchmark with an explicit split policy.

Check	Required behavior
Duplicate genomes	Deduplicate or group close duplicates before splitting
Genome leakage	Avoid near-identical isolates crossing train/test when the evaluation is intended to measure generalization
Phenotype provenance	Use laboratory-measured fields for ground truth
MIC censoring	Preserve < / > signs; never pretend censored measurements are exact
Drug naming	Canonicalize antibiotic names but retain the source spelling
Breakpoint versions	Record CLSI/EUCAST year when converting MIC to categorical labels
Class imbalance	Report per-drug metrics and not accuracy alone
Calibration	Include reliability/Brier-style metrics for probability outputs
Reproducibility	Pin tool/database/model versions

Recommended metrics: AUROC and AUPRC for discrimination; sensitivity/specificity or precision/recall at a documented threshold; Brier score or calibration error for probability quality; and a confusion matrix by antibiotic. For MIC prediction, report MAE/RMSE and interval/censor-aware error summaries where appropriate.

14. The 120 GB Question

What the big dataset claim should mean

Do not describe “120 GB” as the AMRFinderPlus database. NCBI describes AMRFinderPlus as open-source software plus curated reference gene and HMM resources. The large storage footprint is more plausibly a broader genome corpus, organizer corpus, or multi-source cache.

For Umbrella OS, define storage in categories: raw genome sequences, derived annotations, indexes, benchmark tables, and model artifacts. A 120 GB target is therefore a **data lake capacity**, not a single database file.

Storage class	Example contents	Policy
Raw genomes	FASTA / FASTQ / metadata	Immutable, checksummed
Annotations	AMRFinderPlus / ResFinder outputs	Versioned by tool + DB
Indexes	Trie / Bloom / XTree	Rebuildable artifacts
Benchmark	Parquet / CSV / JSON	Pinned, split-aware
Model artifacts	weights / scalars / calibrators	Versioned + hashed

If the organizers explicitly provide or authorize a ~120 GB corpus, build an ingestion manifest and stream the data rather than loading the entire corpus into RAM. Your existing Genome Firewall architecture already demonstrates the right systems principle: memory-mapped or streaming ingestion, immutable indexes, parallel scoring and constant-size auxiliary memory. ■filecite■turn0file0■L59-L76■

15. Hackathon Build Checklist

What to have ready before touching the full corpus

Time	Deliverable	Definition of done
0-1h	Dataset manifest	Files, hashes, schema, split, provenance recorded
1-3h	Ingestion	FASTA + phenotype join works on organizer sample
3-5h	QC	Bad records and duplicates are detected
5-7h	AMR annotation	AMRFinderPlus pipeline returns normalized hits
7-9h	Baseline model	Per-antibiotic classifier trains and evaluates
9-11h	Core API	FastAPI endpoints return analysis + evidence
11-14h	Umbrella apps	Genome / Mutation / AMR / Sample / Analytics views work
14-16h	Vella	Natural-language orchestration across the apps
16-17.5h	Forge	Generate one safe native scientific app
17.5-18.5h	Demo polish	Seed data, visual audit trail, architecture screen, final run

Do not spend the sprint downloading everything. The judging demo needs a deterministic dataset slice, a fast pipeline, and a visible evidence chain. A giant corpus is a post-hackathon scale milestone.

16. Source Directory

Canonical starting points

Resource	URL	Role
BV-BRC home	https://www.bv-brc.org/	Primary bacterial genome and phenotype resource
BV-BRC Genome API	https://www.bv-brc.org/api/doc/genome	Genome metadata/API schema
BV-BRC Genome AMR API	https://www.bv-brc.org/api/doc/genome_amr	Laboratory AMR + MIC fields and provenance
BV-BRC FTP guide	https://www.bv-brc.org/docs/quick_references/ftp.html	FTPS structure and bulk download
BV-BRC AMR Phenotypes	https://www.bv-brc.org/docs/quick_references/organisms_taxon/amr_phenotypes.html	Interactive phenotype table behavior
NCBI AMRFinderPlus	https://www.ncbi.nlm.nih.gov/pathogens/antimicrobial-resistance/AMRFinder/	Official tool + database
NCBI AMR Resources	https://www.ncbi.nlm.nih.gov/pathogens/antimicrobial-resistance/resources/	Reference genes, HMMs, AST resources
AMRFinderPlus source	https://github.com/ncbi/amr	Source, releases, documentation
ResFinder	https://genepi.food.dtu.dk/resfinder	Independent AMR gene/mutation annotation
cAMRah paper	https://pmc.ncbi.nlm.nih.gov/articles/PMC12910510/	Multi-tool harmonized AMR workflow
XTree	https://github.com/two-frontiers-project/2FP-XTree	Fast k-mer reference/search layer
HyenaDNA	https://github.com/HazyResearch/hyena-dna	Long-context genomic foundation model
DNABERT-2	https://github.com/MAGICS-LAB/DNABERT_2	117M multi-species genome model

Verification notes: BV-BRC documentation currently recommends encrypted FTPS for public downloads; the AMRFinderPlus page states that the software and databases are open source and freely available; XTree documentation describes k-mer based sequence lookup/tallying and large-reference support; HyenaDNA and DNABERT-2 project pages document their long-context / foundation-model capabilities. Source facts were checked against current public documentation during preparation of this guide.

17. Data Contract for Umbrella Core

The interface between data science and the OS

```
GenomeRecord
- genome_id
- assembly_accession
- organism
- taxon_id
- fasta_uri
- qc_status

PhenotypeRecord
- genome_id
- antibiotic
- measurement_value
- measurement_sign
- measurement_unit
- resistant_phenotype
- testing_standard
- testing_standard_year
- laboratory_typing_method
- evidence_source

AnnotationRecord
- genome_id
- tool
- tool_version
- database_version
- marker
- mutation
- locus
- evidence

PredictionRecord
- genome_id
- antibiotic
- probability
- class
- model_version
- feature_set
- evidence_refs[]
```

This contract allows the desktop applications to remain independent of the underlying data source. Today the backend may read the organizer benchmark. Tomorrow it can read a BV-BRC snapshot or a local research project without redesigning the UI.

Scientific UX rule: every important result in Umbrella OS should have a “Why?” or “Evidence” panel. Show the source genome, phenotype record, annotation tool/version, relevant marker or feature, model version and assumptions.

End state: the data layer becomes the foundation for Genome Analyzer, Mutation Lab, AMR Sentinel, Sample Vault, Sequence QC, Variant Explorer, Research Desk, Analysis Studio and the AI orchestration layer. The same data contract also makes Umbrella Forge capable of generating small, safe native analysis apps on demand.